Contents

1 Agentic RAG with Contextual AI

Quick facts

Category: Key Collaboration
Type: Agentic / Orchestrated RAG
Key Feature: Tool-using agent + parsing/reranking + grounded generation + checks

What It Is

A production-oriented agentic RAG workflow combining multiple retrieval tools with parsing and verification.

Pseudocode Concept (Python light IDE format)

```
        from pydantic_ai import Agent

        agent = Agent(
            "openai:gpt-4o-mini",
            system_prompt="Choose tools to retrieve strong evidence. Prefer full-doc
                when chunks are insufficient."
        )

        @agent.tool
        async def search_knowledge_base(query: str, limit: int = 8) -> list[dict]:
            """Semantic search over chunks (optionally hybrid with BM25)."""
            q_emb = await embedder.embed_query(query)
            return await db.match_chunks(q_emb, limit)

        @agent.tool
        async def retrieve_full_document(title: str) -> str:
            """Fetch full document text for deep context."""
            doc = await db.get_document_by_title(title)
            return f"{doc['title']}

{doc['content']}"

        async def run(question: str) -> str:
            # 1) initial search
            chunks = await search_knowledge_base(question, limit=8)

            # 2) agent decides if we need full-doc context
            if needs_full_doc(question, chunks):
                title = guess_title(chunks)
                full_doc = await retrieve_full_document(title)
                context = [full_doc]
            else:
                context = chunks

            # 3) rerank + answer
            context = rerank(question, context)[:6]
```

```
36            return await agent.run(format_prompt(question, context))
```

Pros

- Handles complex, multi-step questions
- Flexible retrieval (chunks vs full-doc)
- Verification hooks per step

Cons

- More components to operate
- Higher latency/cost than simple RAG
- Needs strong guardrails

Requirements / Applications

- Enterprise doc QA, customer support, research assistants

Library / Example of Tooling

- PydanticAI/LangChain/LangGraph
- Doc parsers (Docling, unstructured)
- Vector DB + reranker

2 Basic RAG

Quick facts

Category: Foundational
Type: Standard RAG
Key Feature: Top-k retrieval + prompt augmentation

What It Is

Classic baseline: retrieve top-k chunks and prompt the model with that context.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent(
    "openai:gpt-4o-mini",
    system_prompt="Answer using ONLY the provided context. Cite sources."
)

@agent.tool
async def search_knowledge_base(query: str, limit: int = 5) -> list[dict]:
    """Vector search over embedded chunks."""
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, limit)

async def run(question: str) -> str:
    ctx = await search_knowledge_base(question, limit=5)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Fast to build
- Good baseline for evaluation
- Improves factuality vs. pure LLM

Cons

- Sensitive to chunking and retrieval errors
- May miss key evidence
- Can hallucinate if context is weak

Requirements / Applications

- General Q&A over a fixed corpus; FAQs

Library / Example of Tooling

- LangChain/LlamaIndex/Haystack
- FAISS/Chroma/pgvector
- e5/bge/OpenAI embeddings

3 RAG with CSV Files

Quick facts

Category: Foundational
Type: Standard RAG (structured source)
Key Feature: Row-wise indexing + metadata filters

What It Is

RAG over structured CSVs by converting rows (or groups) into retrievable units with metadata.

Pseudocode Concept (Python light IDE format)

```python
import pandas as pd
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Answer using retrieved rows as evidence.")

def row_to_text(row: dict) -> str:
    # Serialize a row into a stable, readable representation
    return "\n".join([f"{k}: {v}" for k, v in row.items()])

async def ingest_csv(path: str):
    df = pd.read_csv(path)
    for i, r in df.iterrows():
        await db.upsert_chunk(
            chunk_id=f"row:{i}",
            content=row_to_text(r.to_dict()),
            metadata={"row": int(i)}
        )

@agent.tool
async def search_rows(query: str, k: int = 6) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def run(question: str) -> str:
    rows = await search_rows(question, k=6)
    return await agent.run(format_prompt(question, rows))
```

Pros

- Strong provenance (row IDs)
- Easy column filtering
- Good for exports/catalogs

Cons

- Schema-to-text design needed
- Joins/relations need extra logic
- Large CSV may require grouping

Requirements / Applications

- Product catalogs, ticket exports, lookup tables

Library / Example of Tooling

- Pandas + DuckDB/SQLite
- CSV loaders
- Vector DB metadata filters

4 Reliable RAG

Quick facts

Category: Foundational
Type: Reliability / Grounding
Key Feature: Validation loop (relevance + grounding checks)

What It Is

A reliability-focused RAG loop that checks relevance/grounding and re-retrieves or refines when needed.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Be grounded. If evidence is weak, retrieve again.")

@agent.tool
async def retrieve(query: str, k: int = 10) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def is_relevant(question: str, ctx: list[dict]) -> bool:
    return await llm.yes_no(f"Is context relevant to: {question}\n\n{summarize(ctx)}")

```

```python
async def is_grounded(answer: str, ctx: list[dict]) -> bool:
    return await llm.yes_no(f"Is the answer fully supported by context?\nA:
        {answer}\nC: {summarize(ctx)}")

async def run(question: str) -> str:
    ctx = await retrieve(question, k=10)

    if not await is_relevant(question, ctx):
        ctx = await retrieve(await llm.rewrite(question), k=15)

    draft = await agent.run(format_prompt(question, ctx))

    if not await is_grounded(draft, ctx):
        draft = await agent.run(refine_prompt(question, ctx, draft))

    return draft
```

Pros

- Lower hallucination risk
- More robust on edge cases
- Better traceability

Cons

- More latency
- Validators imperfect
- Needs thresholds/test set

Requirements / Applications

- High-stakes domains; support automation

Library / Example of Tooling

- LLM-as-judge / heuristics
- Rerankers
- DeepEval/TruLens/LangSmith

5 Optimizing Chunk Sizes

Quick facts

Category: Foundational
Type: Chunking / Indexing
Key Feature: Chunk/overlap tuning with offline eval

What It Is

Tune chunk size/overlap to balance recall, coherence, and token cost using offline evaluation.

Pseudocode Concept (Python light IDE format)

```python
# Chunk size tuning (offline). Evaluate each configuration on a test set.

def tune_chunking(corpus: list[str], queries: list[str], sizes: list[int]) -> dict:
    results = {}
    for sz in sizes:
        chunks = chunk(corpus, size=sz, overlap=sz // 10)
        index = build_vector_index(chunks)
        results[sz] = evaluate(index, queries) #
            accuracy/faithfulness/retrieval-hit-rate
    return pick_best(results)

best = tune_chunking(corpus, eval_queries, sizes=[256, 512, 800, 1200])
```

Pros

- High ROI improvement
- Better recall and coherence
- Reduces token waste

Cons

- Data dependent
- Requires re-indexing
- May overfit

Requirements / Applications

- Any RAG system; run regression tests

Library / Example of Tooling

- Chunkers + eval harness (RAGAS/DeepEval)
- Vector DB

6 Proposition Chunking

Quick facts

Category: Foundational
Type: Chunking / Knowledge extraction
Key Feature: LLM-assisted proposition extraction

What It Is

Split documents into atomic factual propositions for high-precision retrieval.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Answer using proposition-level evidence.")

async def extract_propositions(text: str) -> list[str]:
    """Turn text into atomic factual statements."""
    return await llm.generate_list(f"Extract factual propositions:\n\n{text}")

async def ingest(doc_text: str, doc_id: str):
    props = await extract_propositions(doc_text)
    for i, p in enumerate(props):
        await db.upsert_chunk(
            chunk_id=f"{doc_id}:prop:{i}",
            content=p,
            metadata={"doc_id": doc_id}
        )

@agent.tool
async def search_props(query: str, k: int = 12) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def run(question: str) -> str:
    ctx = await search_props(question, k=12)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- High precision for factual Q&A
- Fine-grained grounding
- Useful for policies/specs

Cons

- Index-time cost
- Extractor quality matters
- May lose narrative flow

Requirements / Applications

- Compliance/policy/specs; fact lookup

Library / Example of Tooling

- LLM proposition extractor + graders
- Vector DB proposition index

7 Query Transformations

Quick facts

Category: Query Enhancement
Type: Query rewriting / decomposition
Key Feature: Query rewriting + decomposition

What It Is

Rewrite/decompose the query into better retrieval queries (faithful paraphrases and sub-questions).

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Rewrite queries without changing intent.")

@agent.tool
async def rewrite_query(query: str) -> list[str]:
    """Generate faithful rewrites + (if needed) decomposed sub-queries."""
```

```python
    return await llm.generate_list(f"Rewrite faithfully and decompose if needed: {query}")

@agent.tool
async def search(query: str, k: int = 10) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def run(question: str) -> str:
    queries = [question] + await rewrite_query(question)

    hits = []
    for q in queries:
        hits.extend(await search(q, k=10))

    ctx = deduplicate(hits)[:8]
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Improves recall on vague queries
- Helps multi-part questions
- No corpus changes needed

Cons

- Can drift without constraints
- More retrieval calls
- Needs guardrails

Requirements / Applications

- Conversational assistants; ambiguous questions

Library / Example of Tooling

- LLM query rewriter
- Hybrid retrievers
- Caching

8 HyDE (Hypothetical Document Embedding)

Quick facts

Category: Query Enhancement
Type: Query-to-document synthesis
Key Feature: Synthetic document embedding

What It Is

HyDE: generate a hypothetical passage, embed it, and retrieve matching real documents.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use HyDE to improve retrieval.")

@agent.tool
async def hyde_document(question: str) -> str:
    """Generate a hypothetical passage that would answer the question."""
    return await llm.generate(f"Write a concise passage answering: {question}")

@agent.tool
async def search_by_text(text: str, k: int = 8) -> list[dict]:
    emb = await embedder.embed_query(text)
    return await db.match_chunks(emb, k)

async def run(question: str) -> str:
    hypo = await hyde_document(question)
    ctx = await search_by_text(hypo, k=8)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Boosts retrieval for short/vague queries
- Helps long-tail questions
- Works with any vector DB

Cons

- Extra LLM call
- Hypothesis can mislead
- Needs safe prompts

Requirements / Applications

- Sparse queries; research Q&A

Library / Example of Tooling

- LLM for HyDE
- Embeddings + vector search
- Optional reranker

9 HyPE (Hypothetical Prompt Embedding)

Quick facts

Category: Query Enhancement
Type: Prompt synthesis for retrieval
Key Feature: Prompt-style query expansion

What It Is

HyPE: generate prompt-like task instructions and embed them to steer retrieval toward task intent.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use HyPE to align retrieval with task intent.")

@agent.tool
async def hype_text(question: str) -> str:
    """Generate short task-focused instructions while preserving intent."""
    return await llm.generate(
        "Create retrieval-oriented instructions for answering this question faithfully:\n"
        f"{question}"
    )

@agent.tool
async def search(query_like_text: str, k: int = 8) -> list[dict]:
    emb = await embedder.embed_query(query_like_text)
    return await db.match_chunks(emb, k)

async def run(question: str) -> str:
    expanded = await hype_text(question)
    ctx = await search(expanded, k=8)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Aligns retrieval with task intent
- Useful for summarize/compare/extract
- Constraint-friendly

Cons

- Extra LLM call
- May over-specify
- Needs tuning

Requirements / Applications

- Task-oriented assistants

Library / Example of Tooling

- LLM prompt synthesizer
- Embeddings + vector search
- Optional router

10 Contextual Chunk Headers

Quick facts

Category: Context Enrichment
Type: Contextualization / Chunk augmentation
Key Feature: Document/section context prefixes

What It Is

Add a section-aware header to each chunk before embedding to reduce ambiguity.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use section-aware chunks.")

def add_header(chunk: str, title: str, heading_path: str) -> str:
    return f"This chunk is from '{title}' (section: {heading_path}).\n\n{chunk}"

```

```python
async def ingest(doc):
    for chunk, heading_path in chunk_with_headings(doc.text):
        await db.upsert_chunk(doc.id, add_header(chunk, doc.title, heading_path))

@agent.tool
async def search(query: str, k: int = 6) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def run(question: str) -> str:
    ctx = await search(question, k=6)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Improves retrieval precision
- Reduces chunk ambiguity
- Great for long docs

Cons

- More tokens
- Bad headers hurt
- Needs consistent headings

Requirements / Applications

- Handbooks/policies/long PDFs

Library / Example of Tooling

- Heading extraction
- Vector DB
- Optional reranker

11 Relevant Segment Extraction

Quick facts

Category: Context Enrichment
Type: Context selection / Extraction
Key Feature: Span-level extraction after retrieval

What It Is

Retrieve docs then extract only the most relevant spans before generation.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Extract only the relevant spans.")

@agent.tool
async def retrieve_docs(query: str, k: int = 8) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_documents(q_emb, k)

async def extract_spans(query: str, doc_text: str) -> str:
    return await llm.generate(f"Extract minimal supporting spans for:
        {query}\n\n{doc_text}")

async def run(question: str) -> str:
    docs = await retrieve_docs(question, k=8)
    spans = [await extract_spans(question, d["content"]) for d in docs]
    return await agent.run(format_prompt(question, spans))
```

Pros

- Higher signal-to-noise
- Fits more evidence in context
- Less irrelevant content

Cons

- Extractor may drop nuance
- Adds latency
- Requires good parsing

Requirements / Applications

- Long docs; strict token budgets

Library / Example of Tooling

- LLM span extractor
- Compression retrievers
- Cross-encoder extractors

12 Context Window Enhancement

Quick facts

Category: Context Enrichment
Type: Sliding-window / Neighbor expansion
Key Feature: Neighbor expansion

What It Is

Expand retrieved context with neighbor chunks around each hit.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use neighbor chunks for coherence.")

@agent.tool
async def search_hits(query: str, k: int = 4) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

@agent.tool
async def fetch_neighbors(doc_id: str, pos: int, window: int = 1) -> list[str]:
    """Fetch previous/next chunks around a hit."""
    return await db.get_chunk_window(doc_id, pos, window)

async def run(question: str) -> str:
    hits = await search_hits(question, k=4)

    ctx = []
    for h in hits:
        ctx.extend(await fetch_neighbors(h["doc_id"], h["pos"], window=1))

    ctx = deduplicate(ctx)[:10]
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Restores local context
- Improves coherence
- Simple to implement

Cons

- Can add noise

- Higher token use
- Needs chunk ordering metadata

Requirements / Applications

- Technical/narrative docs

Library / Example of Tooling

- Chunk position metadata
- Docstore APIs

13 Semantic Chunking

Quick facts

Category: Context Enrichment
Type: Chunking (semantic boundaries)
Key Feature: Boundary-aware chunking

What It Is

Chunk at semantic boundaries (topic shifts) rather than fixed sizes.

Pseudocode Concept (Python light IDE format)

```python
# Semantic chunking: split at topic boundaries instead of fixed windows.

def semantic_chunks(text: str) -> list[str]:
    sentences = split_sentences(text)
    groups = segment_by_similarity(sentences, threshold=0.75)
    return [" ".join(g) for g in groups]

def build_index(docs: list[str]):
    chunks = []
    for d in docs:
        chunks.extend(semantic_chunks(d))
    return build_vector_index(chunks)
```

Pros

- More coherent chunks
- Better conceptual retrieval
- Less fragmentation

Cons

- More complex preprocessing
- Boundary errors possible
- Variable chunk sizes

Requirements / Applications

- Papers/specs/policies

Library / Example of Tooling

- Sentence segmentation + clustering
- LLM-assisted chunking

14 Contextual Compression

Quick facts

Category: Context Enrichment
Type: Compression / Summarization for retrieval
Key Feature: Query-aware compression

What It Is

Compress retrieved context into query-focused summaries/extractions before answering.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Compress context before
    answering.")

@agent.tool
async def retrieve(query: str, k: int = 10) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def compress(query: str, chunk: str) -> str:
    return await llm.generate(
        f"Compress for answering '{query}'. Keep only relevant facts:\n\n{chunk}"
    )

async def run(question: str) -> str:
    raw = await retrieve(question, k=10)
```

```
    compressed = [await compress(question, c["content"]) for c in raw]
    return await agent.run(format_prompt(question, compressed))
```

Pros

- Fits more evidence
- Improves relevance density
- Helps small context windows

Cons

- Compression may drop nuance
- Extra compute
- Attribution complexity

Requirements / Applications

- Long docs + token limits

Library / Example of Tooling

- LangChain compression retrievers
- LLM summarizers

15 Document Augmentation

Quick facts

Category: Context Enrichment
Type: Synthetic data augmentation for retrieval
Key Feature: Index-time augmentation

What It Is

Augment docs at index time with synthetic questions/keywords to improve retrievability.

Pseudocode Concept (Python light IDE format)

```python
# Document augmentation: add synthetic questions/keywords at index time.

async def augment(doc_text: str) -> str:
    qs = await llm.generate_list(f"Generate 5 likely user questions answered by this text:\n\n{doc_text}")
    return doc_text + "\n\nPossible questions:\n" + "\n".join(qs)

async def ingest(doc_id: str, doc_text: str):
    enriched = await augment(doc_text)
    for chunk in chunk_text(enriched, size=800, overlap=80):
        await db.upsert_chunk(chunk_id=f"{doc_id}:{hash(chunk)}", content=chunk)
```

Pros

- Boosts recall
- Helps indirect phrasing
- Good for small KBs

Cons

- Index-time cost
- Synthetic noise risk
- Needs eval

Requirements / Applications

- Sparse corpora; varied user language

Library / Example of Tooling

- LLM question generator
- Vector DB

16 Fusion Retrieval

Quick facts

Category: Advanced Retrieval
Type: Hybrid / Fusion retrieval
Key Feature: Dense + BM25 fusion

What It Is

Fuse dense + BM25 results (e.g., RRF) for robust retrieval.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use evidence. Prefer precise matches.")

@agent.tool
async def dense_search(query: str, k: int = 20) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await vector_db.search(q_emb, k=k)

@agent.tool
async def bm25_search(query: str, k: int = 20) -> list[dict]:
    return await search_engine.bm25(query, k=k)

def rrf(hits_a: list[dict], hits_b: list[dict], k: int = 10) -> list[dict]:
    """Reciprocal Rank Fusion (concept)."""
    scores = {}
    for hits in (hits_a, hits_b):
        for rank, h in enumerate(hits, start=1):
            scores[h["id"]] = scores.get(h["id"], 0.0) + 1.0 / (60 + rank)
    return top_k_by_score(scores, k=k)

async def run(question: str) -> str:
    dense = await dense_search(question, k=20)
    sparse = await bm25_search(question, k=20)
    fused = rrf(dense, sparse, k=10)
    ctx = hydrate_chunks(fused)[:6]
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Strong recall
- Handles exact terms and semantics
- Robust across query types

Cons

- More infra
- Needs tuning
- Higher latency

Requirements / Applications

- Support (error codes), logs, mixed jargon

Library / Example of Tooling

- Elasticsearch/OpenSearch
- Vector DB
- RRF fusion

17 Reranking

Quick facts

Category: Advanced Retrieval
Type: Ranking / Relevance modeling
Key Feature: Second-stage reranking

What It Is

Retrieve many candidates then rerank with a stronger model before answering.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Ground answers in context.")

@agent.tool
async def vector_search(query: str, candidate_limit: int = 20) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, candidate_limit)

async def rerank(query: str, candidates: list[dict], top_k: int = 5) -> list[dict]:
    """Second-stage rerank using a cross-encoder/LLM scorer."""
    pairs = [[query, c["content"]] for c in candidates]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
    return [c for c, _ in ranked[:top_k]]

async def run(question: str) -> str:
    candidates = await vector_search(question, candidate_limit=20)
    ctx = await rerank(question, candidates, top_k=5)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Big precision gain
- Less irrelevant context
- Better grounded answers

Cons

- Extra cost/latency
- Needs candidate set
- Reranker infra

Requirements / Applications

- Precision-critical Q&A

Library / Example of Tooling

- bge-reranker/cross-encoders/Cohere
- Caching

18 Multi-faceted Filtering

Quick facts

Category: Advanced Retrieval
Type: Metadata & rule-based filtering
Key Feature: Metadata-aware retrieval

What It Is

Apply ACL/tenant/time/product filters pre/post retrieval.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Respect metadata filters (ACL, tenant, product).")

@agent.tool
async def search(query: str, k: int, where: dict) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k, where=where)
```

```python

async def run(question: str, user) -> str:
    where = {"tenant": user.tenant, "role": user.role, "product": user.product}
    ctx = await search(question, k=8, where=where)
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Better relevance
- Compliance & access control
- Personalization

Cons

- Requires clean metadata
- Over-filtering hurts recall
- Ingestion complexity

Requirements / Applications

- Enterprise multi-tenant RAG

Library / Example of Tooling

- Vector DB metadata filters
- RBAC/ABAC

19 Hierarchical Indices

Quick facts

Category: Advanced Retrieval
Type: Hierarchical retrieval (coarse→fine)
Key Feature: Coarse→fine retrieval

What It Is

Hierarchical indices (doc→section→chunk) with coarse-to-fine retrieval.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Coarse-to-fine retrieval over hierarchical indices.")

@agent.tool
async def doc_search(query: str, k: int = 5) -> list[dict]:
    return await doc_index.search(query, k=k)

@agent.tool
async def section_search(query: str, doc_ids: list[str], k: int = 10) -> list[dict]:
    return await section_index.search(query, doc_ids=doc_ids, k=k)

@agent.tool
async def chunk_search(query: str, section_ids: list[str], k: int = 12) -> list[dict]:
    return await chunk_index.search(query, section_ids=section_ids, k=k)

async def run(question: str) -> str:
    docs = await doc_search(question, k=5)
    secs = await section_search(question, [d["id"] for d in docs], k=10)
    chunks = await chunk_search(question, [s["id"] for s in secs], k=12)
    return await agent.run(format_prompt(question, chunks[:6]))
```

Pros

- Scales to large corpora
- Better coherence
- Improves coverage

Cons

- More indexing work
- More steps
- Hierarchy mapping needed

Requirements / Applications

- Large doc sets

Library / Example of Tooling

- LlamaIndex hierarchical nodes
- Docstore mapping

20 Ensemble Retrieval

Quick facts

Category: Advanced Retrieval
Type: Retriever ensembling
Key Feature: Retriever ensemble

What It Is

Ensemble multiple retrievers (dense/sparse/domain) and merge results.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Ensemble multiple retrievers and merge results.")

@agent.tool
async def dense_search(query: str, k: int = 10) -> list[dict]:
    return await dense_retriever.search(query, k=k)

@agent.tool
async def sparse_search(query: str, k: int = 10) -> list[dict]:
    return await bm25.search(query, k=k)

@agent.tool
async def domain_search(query: str, k: int = 10) -> list[dict]:
    return await domain_retriever.search(query, k=k)

async def run(question: str) -> str:
    hits = []
    for tool in (dense_search, sparse_search, domain_search):
        hits.extend(await tool(question, k=10))

    ctx = deduplicate(hits)[:8]
    return await agent.run(format_prompt(question, ctx))
```

Pros

- Robustness across query styles
- Improved recall
- Reduces blind spots

Cons

- More infra/tuning

- Duplicates/noise
- Latency

Requirements / Applications

- Heterogeneous corpora

Library / Example of Tooling

- Multiple embedding models
- BM25 + dense
- RRF/weighted merge

21 Dartboard Retrieval

Quick facts

Category: Advanced Retrieval
Type: Diversified retrieval / Coverage-first
Key Feature: Diversified retrieval

What It Is

Diversified retrieval to maximize topical coverage and reduce redundancy.

Pseudocode Concept (Python light IDE format)

```python
# Dartboard/Diversified retrieval (concept): pick results that maximize coverage.

def mmr_select(candidates: list[dict], k: int) -> list[dict]:
    selected = []
    while len(selected) < k:
        nxt = argmax_mmr(candidates, selected)
        selected.append(nxt)
    return selected

async def run(question: str) -> str:
    candidates = await retriever.search(question, k=50)
    ctx = mmr_select(candidates, k=10)
    return await llm.generate(answer_prompt(question, ctx))
```

Pros

- Better coverage for broad queries
- Less redundancy
- Helps exploration

Cons

- May reduce precision for narrow queries
- Needs diversity scoring
- More complexity

Requirements / Applications

- Exploratory research queries

Library / Example of Tooling

- MMR
- Clustering + sampling

22 Multi-modal RAG with Captioning

Quick facts

Category: Advanced Retrieval
Type: Multimodal RAG
Key Feature: Caption/vision embeddings + retrieval

What It Is

Multimodal RAG by captioning images and retrieving across text+image evidence.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Answer using both text and image evidence.")

async def ingest_image(img):
    caption = await vision.caption(img) # or multimodal embedding
    await db.upsert_chunk(
        chunk_id=f"img:{img.id}",
```

```
        content=caption,
        metadata={"image_id": img.id}
    )

@agent.tool
async def search(query: str, k: int = 6) -> list[dict]:
    q_emb = await embedder.embed_query(query)
    return await db.match_chunks(q_emb, k)

async def run(question: str) -> str:
    hits = await search(question, k=6) # returns captions + image_ids
    images = [h["metadata"]["image_id"] for h in hits if "image_id" in h["metadata"]]
    return await mm_llm.answer_with_images(question, hits, images)
```

Pros

- Works on diagrams/screenshots
- Unlocks visual knowledge
- Better PDFs/slides

Cons

- Caption quality limits retrieval
- More compute/storage
- Needs vision models

Requirements / Applications

- Manuals, slides, screenshots

Library / Example of Tooling

- BLIP/GPT-4o captioning
- CLIP embeddings
- Vector DB + media store

23 Retrieval with Feedback Loop

Quick facts

Category: Iterative Techniques
Type: Online learning / Feedback
Key Feature: Feedback loop

What It Is

Use user feedback to improve retrieval/ranking over time.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Log feedback to improve retrieval over time.")

@agent.tool
async def retrieve(query: str, k: int = 6) -> list[dict]:
    return await retriever.search(query, k=k)

async def run(question: str, user_id: str) -> dict:
    ctx = await retrieve(question, k=6)
    answer = await agent.run(format_prompt(question, ctx))
    interaction_id = await logs.store(question, ctx, answer, user_id)
    return {"answer": answer, "interaction_id": interaction_id}

async def on_feedback(interaction_id: str, label: str):
    await logs.store_feedback(interaction_id, label)
    await trainer.maybe_update_reranker()
```

Pros

- Improves with usage
- Aligns with real intent
- Finds KB gaps

Cons

- Needs feedback UX
- Noisy labels risk
- Governance required

Requirements / Applications

- Support assistants

Library / Example of Tooling

- Feedback store
- Reranker fine-tuning
- Analytics

24 Adaptive Retrieval

Quick facts

Category: Iterative Techniques
Type: Routing / Strategy selection
Key Feature: Routing / strategy selection

What It Is

Route queries to different strategies by intent/domain.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Route queries to the best retrieval strategy.")

async def route(query: str) -> str:
    return await llm.classify(query, labels=["policy", "troubleshoot", "definition", "other"])

async def run(question: str) -> str:
    kind = await route(question)
    if kind == "policy":
        ctx = await acl_retriever.search(question, k=8)
    elif kind == "troubleshoot":
        ctx = await hybrid_retriever.search(question, k=10)
    else:
        ctx = await retriever.search(question, k=6)

    return await agent.run(format_prompt(question, ctx))
```

Pros

- Better across mixed workloads
- Lower cost for simple queries
- More consistent quality

Cons

- Routing errors hurt
- More engineering
- Needs monitoring

Requirements / Applications

- Multi-domain assistants

Library / Example of Tooling

- Intent classifier
- Router (LangChain/LlamaIndex)

25 Iterative Retrieval

Quick facts

Category: Iterative Retrieval
Type: Multi-step retrieval
Key Feature: Iterative retrieval

What It Is

Multi-round retrieve→analyze→retrieve loop until evidence is sufficient.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Iteratively retrieve until evidence is sufficient.")

@agent.tool
async def retrieve(query: str, k: int = 8) -> list[dict]:
    return await retriever.search(query, k=k)

async def followup_query(original: str, ctx: list[dict]) -> str | None:
    return await llm.generate_optional(
        "Given the question and context, propose ONE follow-up query if needed.\n"
        f"Q: {original}\nContext: {summarize(ctx)}"
    )

async def run(question: str, max_rounds: int = 3) -> str:
    ctx = []
    q = question

    for _ in range(max_rounds):
        ctx.extend(await retrieve(q, k=8))
        q2 = await followup_query(question, ctx)
        if not q2:
            break
        q = q2
```

```
25
26      ctx = deduplicate(ctx)[:12]
27      return await agent.run(format_prompt(question, ctx))
```

Pros

- Strong on complex questions
- Fills missing evidence
- Robust vs one-shot

Cons

- Higher latency/cost
- Needs stop criteria
- Risk of loops

Requirements / Applications

- Research; RCA; multi-hop Q&A

Library / Example of Tooling

- Agent loop
- Caching
- Query generator

26 DeepEval

Quick facts

Category: Evaluation
Type: RAG evaluation framework
Key Feature: RAG testing harness

What It Is

Automated RAG evaluation harness for metrics and regression testing.

Pseudocode Concept (Python light IDE format)

```python
# DeepEval-style evaluation (concept)

def evaluate_rag(rag, dataset):
    scores = []
    for ex in dataset:
        pred = rag.answer(ex.question)
        scores.append({
            "correctness": correctness(pred, ex.reference),
            "faithfulness": faithfulness(pred, ex.context),
            "relevancy": relevancy(ex.question, pred),
        })
    return aggregate(scores)

report = evaluate_rag(rag_system, golden_dataset)
```

Pros

- Measurable improvements
- Regression protection
- CI-friendly

Cons

- Needs test set
- Judge noise risk
- Cost if LLM-judge

Requirements / Applications

- Production RAG evaluation

Library / Example of Tooling

- DeepEval
- Golden datasets
- CI integration

27 GroUSE

Quick facts

Category: Evaluation
Type: Grounded generation evaluation
Key Feature: Grounded evaluation + judge validation

What It Is

Grounded evaluation focusing on support/grounding and judge reliability checks.

Pseudocode Concept (Python light IDE format)

```python
# GroUSE-style grounded evaluation (concept)

def grounded_metrics(answer: str, context: str) -> dict:
    return {
        "groundedness": groundedness(answer, context),
        "support": evidence_support(answer, context),
        "judge_reliability": judge_consistency(answer, context),
    }

metrics = grounded_metrics(answer, retrieved_context)
```

Pros

- More focus on grounding
- Improves evaluator reliability
- Useful for high-stakes apps

Cons

- Complex eval setup
- Depends on datasets
- Compute cost

Requirements / Applications

- Regulated domains

Library / Example of Tooling

- GroUSE
- LLM judge unit tests

28 Explainable Retrieval

Quick facts

Category: Explainability
Type: Attribution / Transparency
Key Feature: Attribution / provenance

What It Is

Show why retrieval happened and which evidence supports the answer (provenance).

Pseudocode Concept (Python light IDE format)

```python
# Explainable retrieval (concept)

async def run(question: str) -> dict:
    hits = await retriever.search_with_scores(question, k=8)

    # 1) show why each hit was selected (scores + matched terms + metadata)
    why = explain_scores(question, hits)

    # 2) answer with citations/spans
    answer = await llm.generate(answer_with_citations_prompt(question, hits))

    return {"answer": answer, "evidence": hits, "why": why}
```

Pros

- Improves trust
- Easier debugging
- Audit-friendly

Cons

- Extra UX work
- Explanation noise
- Sensitive metadata risk

Requirements / Applications

- Enterprise assistants with human review

Library / Example of Tooling

- Citation spans
- Tracing (OpenTelemetry)

29 Graph RAG with LangChain

Quick facts

Category: Advanced Architecture
Type: Graph-augmented retrieval
Key Feature: Graph traversal + vector search

What It Is

Graph + vector retrieval for relationship-heavy multi-hop questions.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use graph traversal + retrieval for multi-hop questions.")

@agent.tool
async def extract_entities(question: str) -> list[str]:
    return await llm.generate_list(f"List key entities in: {question}")

@agent.tool
async def graph_neighbors(entities: list[str], hops: int = 2) -> list[str]:
    seeds = await graph.lookup_nodes(entities)
    return await graph.traverse(seeds, hops=hops)

@agent.tool
async def retrieve_from_nodes(question: str, node_ids: list[str], k: int = 10) -> list[dict]:
    return await vector_db.search(question, k=k, where={"node_id": node_ids})

async def run(question: str) -> str:
    ents = await extract_entities(question)
    nodes = await graph_neighbors(ents, hops=2)
    ctx = await retrieve_from_nodes(question, nodes, k=10)
    return await agent.run(format_prompt(question, ctx[:6]))
```

Pros

- Strong multi-hop performance
- Entity consistency
- Reduced context sprawl

Cons

- Graph maintenance
- Entity linking quality
- Extra infra

Requirements / Applications

- Interconnected knowledge

Library / Example of Tooling

- Neo4j/NetworkX
- Graph tools + vector DB

30 Microsoft GraphRAG

Quick facts

Category: Advanced Architecture
Type: Community + graph summarization
Key Feature: Community summaries + graph retrieval

What It Is

GraphRAG-style community summaries + local retrieval for global questions.

Pseudocode Concept (Python light IDE format)

```python
# Microsoft GraphRAG-style (concept): combine community summaries + local retrieval.

async def run(question: str) -> str:
    communities = await graph.select_communities(question)
    summaries = await community_store.fetch_summaries(communities)

    local_hits = await retriever.search(question, k=8)
    ctx = summaries + local_hits

    return await llm.generate(answer_prompt(question, ctx))
```

Pros

- Good for corpus-wide questions
- Reusable summaries
- Scales better

Cons

- Heavier preprocessing
- Summary abstraction errors
- Tuning needed

Requirements / Applications

- Large corpora; thematic questions

Library / Example of Tooling

- Microsoft GraphRAG
- Entity extraction pipeline

31 RAPTOR

Quick facts

Category: Advanced Architecture
Type: Tree-organized retrieval (recursive summaries)
Key Feature: Tree index of summaries

What It Is

RAPTOR recursive summarization tree enabling multi-level retrieval.

Pseudocode Concept (Python light IDE format)

```python
# RAPTOR-style (concept): retrieve from a summary tree, then drill down.

async def run(question: str) -> str:
    node = await tree_index.search(question, level="summary", k=1)
    leaves = await tree_index.drill_down(node, question, max_leaves=10)
    return await llm.generate(answer_prompt(question, leaves))
```

Pros

- Efficient on long docs
- High-level + drill-down
- Better coverage

Cons

- Index build cost
- Summary errors propagate
- Complexity

Requirements / Applications

- Long reports/papers

Library / Example of Tooling

- Recursive summarizers
- Tree index patterns

32 Self-RAG

Quick facts

Category: Advanced Architecture
Type: Self-critique + retrieval decision
Key Feature: Self-critique + retrieval decision

What It Is

Self-RAG: decide whether to retrieve and verify groundedness before finalizing.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Decide when to retrieve; verify groundedness.")

async def should_retrieve(question: str) -> bool:
    return await llm.yes_no(f"Do we need documents to answer? {question}")

async def grounded(answer: str, ctx: list[dict]) -> bool:
```

```
    return await llm.yes_no(
        f"Is the answer fully supported by context?\nA: {answer}\nC:
            {summarize(ctx)}"
    )

async def run(question: str) -> str:
    if not await should_retrieve(question):
        return await agent.run(question)

    ctx = await retriever.search(question, k=8)
    draft = await agent.run(format_prompt(question, ctx))

    if not await grounded(draft, ctx):
        ctx2 = await retriever.search(await llm.rewrite(question), k=12)
        draft = await agent.run(format_prompt(question, ctx2))

    return draft
```

Pros

- Avoids unnecessary retrieval
- Improves groundedness
- Catches weak support

Cons

- Extra steps
- Self-checks imperfect
- Latency

Requirements / Applications

- Mixed workloads; hallucination-sensitive

Library / Example of Tooling

- Self-check prompts
- Retrievers + graders

33 Corrective RAG (CRAG)

Quick facts

Category: Advanced Architecture
Type: Corrective retrieval + fallback
Key Feature: Corrective retrieval + fallback

What It Is

CRAG: detect weak retrieval, correct with rewrite + fallback sources/indices, then answer.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Correct low-quality retrieval before answering.")

async def retrieval_score(question: str, ctx: list[dict]) -> float:
    return await llm.score(f"Score retrieval quality 0-1.\nQ: {question}\nC: {summarize(ctx)}")

async def run(question: str) -> str:
    ctx = await retriever.search(question, k=8)
    score = await retrieval_score(question, ctx)

    if score < 0.6:
        q2 = await llm.rewrite_for_search(question)
        ctx = merge(ctx, await retriever.search(q2, k=12))

        # Optional fallback if allowed:
        # ctx = merge(ctx, await web_search(q2))

    return await agent.run(format_prompt(question, ctx[:10]))
```

Pros

- Robust to KB gaps
- Recovers from poor retrieval
- Better reliability

Cons

- More components
- Fallback may be restricted
- Attribution complexity

Requirements / Applications

- Support; incomplete/outdated KB

Library / Example of Tooling

- Retrieval evaluator
- Query rewrite
- Secondary index/web (optional)

34 Sophisticated Controllable Agent

Quick facts

Category: Special Technique
Type: Deterministic agentic RAG
Key Feature: Deterministic control graph + verification

What It Is

Controllable agent driven by a deterministic decision graph with verification and re-planning.

Pseudocode Concept (Python light IDE format)

```python
from pydantic_ai import Agent

agent = Agent("openai:gpt-4o-mini", system_prompt="Use a deterministic control graph + verification.")

class State:
    def __init__(self, question: str):
        self.question = question
        self.tasks = []
        self.answers = []

async def plan(question: str) -> list[dict]:
    """Return ordered sub-tasks with explicit retrieval queries."""
    return await llm.generate_json(f"Decompose into steps with retrieval queries: {question}")

async def verify(answer: str, evidence: list[dict]) -> bool:
    return await llm.yes_no(f"Is the answer supported by evidence?\nA: {answer}\nE: {summarize(evidence)}")

async def run(question: str) -> str:
    s = State(question)
    s.tasks = await plan(question)

    for t in s.tasks:
        evidence = await retriever.search(t["query"], k=10)
        draft = await agent.run(format_prompt(t["task"], evidence))
```

```
25
26          if not await verify(draft, evidence):
27              refined = await retriever.search(await llm.rewrite(t["query"]), k=15)
28              draft = await agent.run(format_prompt(t["task"], refined))
29
30          s.answers.append(draft)
31
32      return await llm.generate(final_synthesis_prompt(question, s.answers))
```

Pros

- More predictable than free-form agents
- Auditable steps
- Strong for complex tasks

Cons

- Design effort
- Higher latency/cost
- Needs robust verifiers

Requirements / Applications

- Reproducible/auditable enterprise workflows

Library / Example of Tooling

- LangGraph/custom FSM
- Vector DB + reranker
- Verifier prompts/tests

References

- Nir Diamant, *RAG_Techniques* (technique list and notebooks). https://github.com/NirDiamant/RAG_Techniques
- Cole Medin, *oTTomator Agents: All RAG Strategies Explained* (pseudocode style inspiration). https://github.com/coleam00/ottomator-agents/tree/main/all-rag-strategies